to_sym → Symbol
: Expresses the receiver as a symbol. This is used by the interpreter when compiling instruction sequences. It's rarely necessary in user code, because the most common case—String to Symbol, is often handled in syntax as :"a weird symbol".

Note that Integer implements the to_int method, and String implements to_str. That way, you can call the strict conversion functions polymorphically:

```
# it doesn't matter if obj is a Fixnum or a
# Roman number, the conversion still succeeds
num = obj.to_int
```

Separately, the Kernel module defines a handful of methods that are available anywhere and act as general conversion methods. These methods are distinctive in Ruby because they begin with capital letters. These methods are generally meant to be the definitive conversions to use when you are just using conversions, rather than defining them.

Array(obj)	Attempts to call to_ary on the argument (this means that if the argument is already an array, it is returned as-is). If the argument doesn't respond to to_ary, it calls to_a. If the argument responds to neither, the argument is returned as the element in a one-element array.
Complex(real, imag, ex: true)	If the argument is a string, calls to_c on the string. Otherwise, creates a complex number that is real + imag * i. If either argument is nil and ex is true, raises a TypeError, if ex is false, then returns nil.
Float(obj, ex: true)	If the obj is numeric, Ruby's numeric conversion is performed. Otherwise, to_f is called on the object. An invalid string will result in an ArgumentError, while a nil object will generate a TypeError. Again, if ex is false, nil is returned rather than throwing exceptions.
Hash(obj)	Calls obj.to_hash.
Integer(obj, base=0, ex: true)	If obj is numeric, Ruby's numeric conversions are performed. Other objects attempt to_int, then to_i. String behavior is slightly different than String#to_i, in that if base is 0, then base prefixes, like 0x, will be used to determine the base. Exception behavior is the same as Float.
Rational(num, den=1, ex: true)	If there's only a string argument, calls to_r on the string. Otherwise returns num/den as a rational. Exception behavior is the same as Float.
String(obj)	Calls obj.to_str if it exists, then tries obj.to_s.

The Symbol.to_proc Trick

Ruby implements the to_proc method for objects of class Symbol. Say you want to convert an array of strings to uppercase. You could write this:

```
names = %w[ant bee cat]
result = names.map { |name| name.upcase }
```

That's fairly concise, right? Return a new array where each element is the corresponding element in the original, converted to uppercase. But you can instead write this:

```
names = %w[ant bee cat]
result = names.map(&:upcase)
```

Now that's concise: apply the upcase method to each element of names.